AI600 Assignment 3 Report

25280048

GitHub Repository:
`https://github.com/Rizi2001/msai_deeplearning_a3`

1 Analytical

This section presents the handwritten solutions for the analytical component of the assignment. It covers the forward and backward pass of the Toy-CNN model, along with conceptual analysis of pooling and normalization techniques.

M. Rizwan Karim 25280048 day / date: A3

Analytical Section

$$X = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 2 \end{bmatrix}, \quad W = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}, \quad b = 0$$

$$v = \begin{bmatrix} 1 & -1 & 2 & 0.5 \end{bmatrix}, \quad c = 1$$

Target (T) = 3 → MSB

(1) (a)

$$\begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 2 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$$

$$= 1.1 + 2.0 + 0.(-1) + 1.1 =$$
$$= 1 + 0 + 0 + 1 = 2$$

$$\begin{bmatrix} 2 & 0 \\ 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$$

$$2 \times 1 + 0 + 1 \times -1 + 1 \times 1 =$$
$$2 + 0 - 1 + 1 = 2$$

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$$

$$0 \times 1 + 1 \times 0 + 1 \times -1 + 0 \times 1 = -1$$

$$\begin{bmatrix} 1 & 1 \\ 0 & 2 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$$

$$1 \times 1 + 1 \times 0 + 0 \times -1 + 2 \times 1 = 3$$

Pre-activation = $\boxed{Z = \begin{bmatrix} 2 & 2 \\ -1 & 3 \end{bmatrix}}$

KAGHAZ www.kaghaz.pk

ReLU(Z) → A $\quad \max(0,z) = \begin{bmatrix} 2 & 2 \\ 0 & 3 \end{bmatrix}$

Flatten $\quad \boxed{A_{flat} = [2,2,0,3]} \quad ; \quad V = [1, -1, 2, 0.5]$

F.C → Y $\quad Y = V \cdot A_{flat} + C$

$= 1\times2 + -1\times2 + 2\times0 + 0.5\times3 + 1$

$= 2 - 2 + 0 + 1.5 + 1$

$\boxed{Y = 2.5}$

M.S.E

Loss → L

$L = \frac{1}{2}(Y-T)^2 = \frac{1}{2}(2.5-3)^2$

$= \frac{1}{2}(0.25)$

$\boxed{L = 0.125}$

(b) Backward pass

Loss Gradient : $\frac{\partial L}{\partial Y} = (Y-T) = 2.5-3 = -0.5$

$\frac{\partial L}{\partial V} = (Y-T) \times A_{flat} = -0.5 \cdot [2,2,0,3]$

$\frac{\partial L}{\partial V} = [-1, -1, 0, -1.5]$

$\frac{\partial L}{\partial A} = (Y-T) \cdot V = -0.5 \cdot [1, -1, 2, 0.5]$

$= [-0.5, 0.5, -1, -0.25]$

Unflatten

↳ $\begin{bmatrix} -0.5 & 0.5 \\ -1 & -0.25 \end{bmatrix} \Rightarrow$ ReLU

$\frac{\partial L}{\partial Z} = \begin{bmatrix} -0.5 & 0.5 \\ 0 & -0.25 \end{bmatrix}$

KAGHAZ
www.kaghaz.pk

day / date:

$\frac{\partial L}{\partial w}$ → Recompute Matrix with updated filter.

$$\frac{\partial L}{\partial z} = \begin{bmatrix} -0.5 & 0.5 \\ 0 & -0.25 \end{bmatrix} \quad X = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 2 \end{bmatrix}$$

a_{11} $1(-0.5) + 2(0.5) + 0(0) + 1(-0.25) = -0.75$

a_{12} $2(-0.5) + 0 + 0 + 1(-0.25) = -1.25$

a^{21} $0 + 0.5 + 0 + 0 = 0.5$

a^{22} $-0.5 + 0.5 + 0 - 0.5 = -0.5$

$$\frac{\partial L}{\partial w} = \begin{bmatrix} -0.75 & -1.25 \\ 0 & -0.5 \end{bmatrix}$$

(c) Weight with largest magnitude → w_{12} → -1.25

→ This will change the most in gradient descent. $\Delta w = w - \eta(\frac{x}{z})$

→ This weight participates in multiple high-activation patches where it had positive activations after ReLU and it would accumulate the strongest influence as large gradient signals from FC layer.

KAGHAZ www.kaghaz.pk

day / date:

(3) (a) $A = \begin{bmatrix} 2 & 2 \\ 0 & 3 \end{bmatrix}$

$Y = \max(A) = 3$

→ Only max elements → 3 get gradient.
→ All other gradients would be zero.
↳ Not affecting those regions in X.

(b) Mean $= \mu = \frac{2+2+0+3}{4} = 1.75$

variance $= \sigma^2 = \frac{(2-1.75)^2 + (2-1.75)^2 + (0-1.75)^2 + (3-1.75)^2}{4}$

$= 1.1875$

Normalized Output $= A_{norm} = \frac{A - \mu}{\sqrt{\sigma^2}}$

$A_{norm} = \begin{bmatrix} 0.23 & 0.23 \\ -1.61 & 1.15 \end{bmatrix}$

(c) Max pooling is concerned with the maximum value, not the position.

→ Same max value may exist even if the position is changed, causing no effect on Y or L.

2 Programming: Implementation and Analysis

2.1 Task 1: Custom CNNs and Shortcut Learning

2.1.1 Part A: Standard MNIST

Architecture Design A custom CNN was implemented with:

- 3 Convolutional Layers
- 2 Fully Connected Layers
- Total parameters $\approx 25,000$ (below 50,000 constraint)

Training and Evaluation The model was trained using Adam optimizer and CrossEntropy loss. Training and validation curves are shown below.

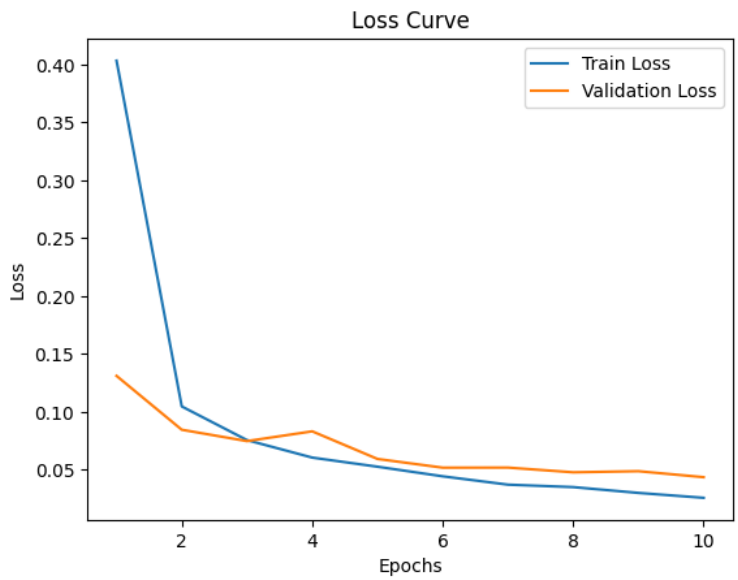

Figure 1: Training and Validation Loss

Final Test Accuracy: **98.84%**

Q1.1 The model generalizes well with minimal overfitting.

Both training and validation loss decrease steadily, and the gap between them remains small throughout training. Validation accuracy closely follows training accuracy (98–99%), indicating good generalization.

A slight gap appears in later epochs (train loss continues decreasing while validation stabilizes), suggesting minor overfitting, but it is not significant.

Q1.2 The filters show basic edge and texture detection.

Several filters exhibit strong intensity transitions corresponding to horizontal and vertical edges with diagonal patterns and local contrast changes.

These filters capture low level spatial features such as edges and strokes, which are essential building blocks for recognizing digit shapes.

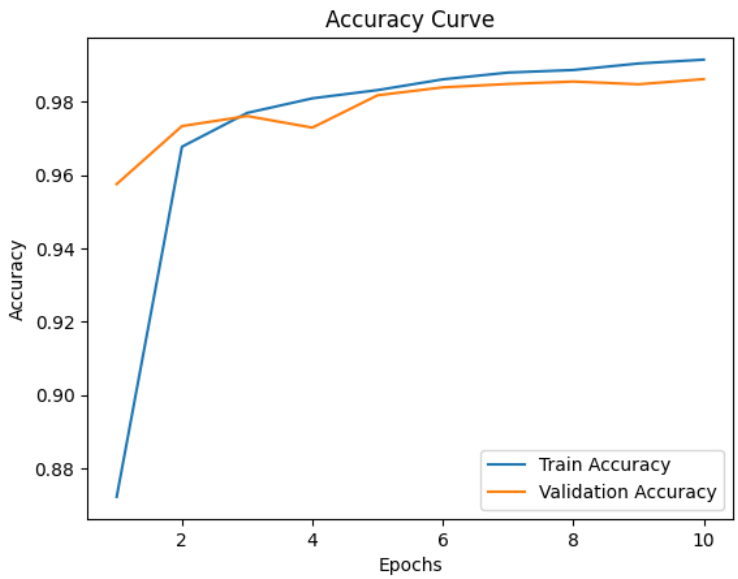

Figure 2: Training and Validation Accuracy

2.1.2 Part B: Colored-MNIST (C-MNIST)

Training and Evaluation Biased Test Accuracy: **99.39%**
Unbiased Test Accuracy: **91.99%**

Q1.3 The model achieves 99.39% accuracy on the biased test set and 91.99% on the unbiased test set, showing a clear performance drop.

This occurs because the model learns a function closer to:

$Y \approx f(color)$ instead of $Y \approx f(shape)$

Mathematically, during training, the optimizer updates weights based on gradients:

Color is highly correlated with labels in the biased dataset which produces stronger, more consistent gradients and faster loss minimization.

As a result, the model prioritizes color (low-complexity feature) over shape (higher-complexity feature).

Conceptually, this is shortcut learning:

The model exploits an easy feature e.g is color here

Q1.4 To force the model to focus on shape instead of color:

- Data augmentation: Randomize or vary colors across samples
- Remove correlation: Ensure color is not predictive of labels during training
- Convert to grayscale: Eliminate color information entirely
- Regularization: Penalize reliance on specific channels (e.g., dropout, weight decay)
- Balanced dataset design: Each digit appears with multiple colors uniformly
- Adversarial training: Encourage invariance to color perturbations

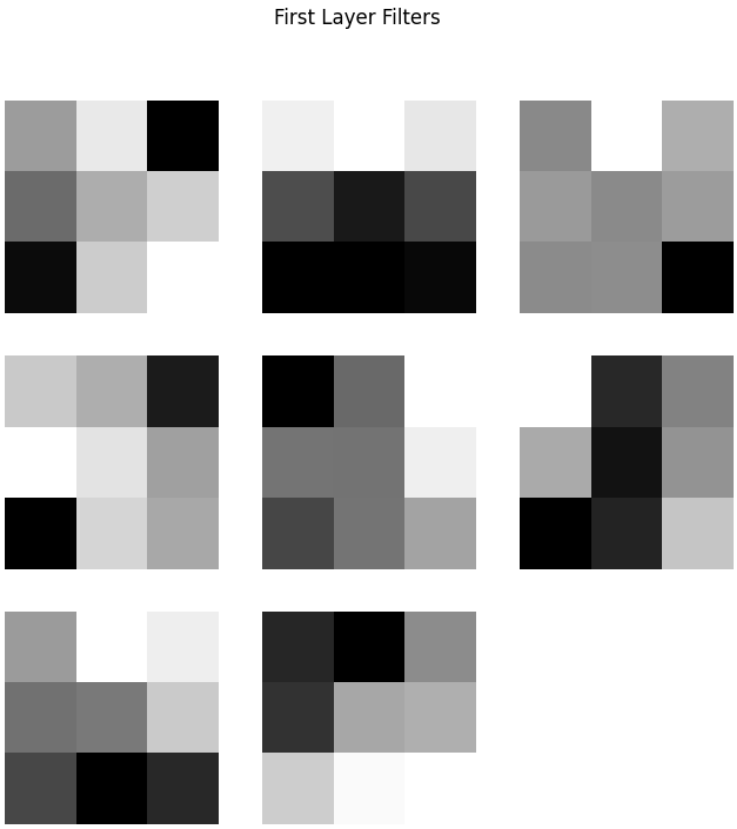

Figure 3: First Layer Filters

2.2 Task 2: Transfer Learning and Interpretability

2.2.1 Part A: Fine-tuning ResNet-18

The ResNet-18 model was loaded with pretrained weights. The backbone was frozen, and only the final fully connected layer was trained.

Final Test Accuracy: **94.27%**

Q2.1 Freezing the early layers is both computationaly efficient and effective because as the trainable parameters are reduced. During training, gradients are computed only for the final classification layer rather than the entire network. This leads to faster training, lower memory consumption and a stable optimization.

This efficiency is evident in the training process, where the model quickly converges within a few epochs, achieving a training accuracy of 94.9% and a test accuracy of 94.27%, despite only updating a small portion of the network.

Since STL-10 is relatively small, training a deep network from scratch would likely result in overfitting due to insufficient data. By freezing the early layers, the model leverages robust, pre-learned feature representations from large-scale datasets, while only adapting the final classification layer to the new task.

This approach allows the model to retain strong general visual features and avoid overfitting.

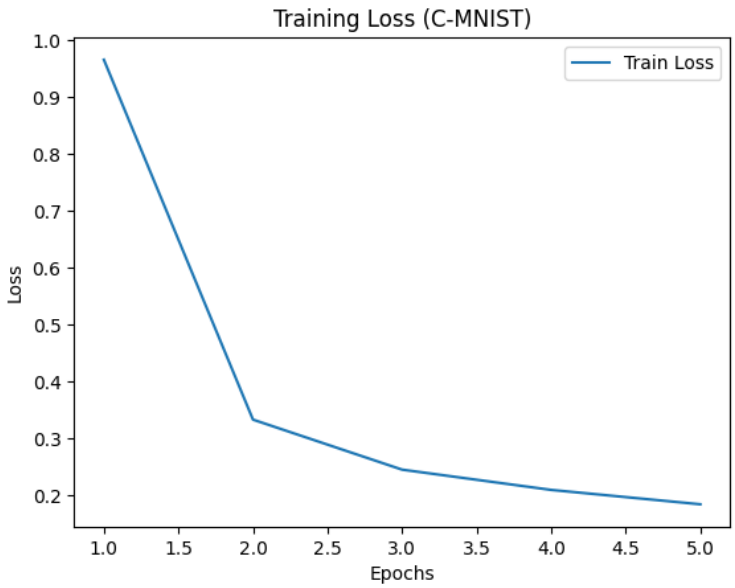

Figure 4: Training Loss (C-MNIST)

2.2.2 Part B: GradCAM Visualization

Q2.2 This visualization shows that for the correctly marked images, the model is focusing the main object of interest rather than the background, so it can check the main features.

In the first image, the heatmap focuses on the horse's body and torso, around the central mass and legs. This indicates that the model is attending to the structural features of the horse, such as shape and posture, which are essential for correct classification.

In the second image, the heatmap is focused on the bird itself while not. much focused on the surrondings. This suggests that the model is successfully isolating the object from its context and focusing on discriminative features such as shape and texture.

This shows model is focusing on good semantically meaningful regions and shows good model learning w.r.t to features

Q2.3 In this case, the model is focusing on misleading or incomplete regions.

The first image is of a truck but classifed as a bird because the heatmap is concentrated on the front wheel and lower portion of the vehicle, rather than the full structure of the truck. By focusing on a localized circular shape and metallic texture, the model fails to capture the global context of the object. Showing that model relied on partial features for this one.

In the second image, (true: deer, predicted: horse), the heatmap highlights primarily the body of the animal, but does not strongly emphasize distinguishing features such as the antlers. Since deer and horses share similar body structures, ignoring key discriminative features leads to confusion between the two classes.

So, in these cases the model attends to localized and partial features and ignored the class specific features like the antlers. Which resulted in incorrect interpretation of the scene.

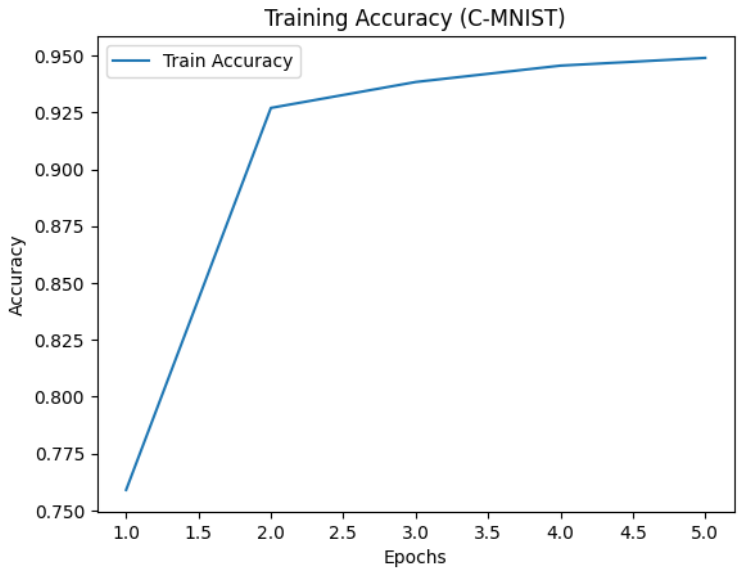

Figure 5: Training Accuracy (C-MNIST)

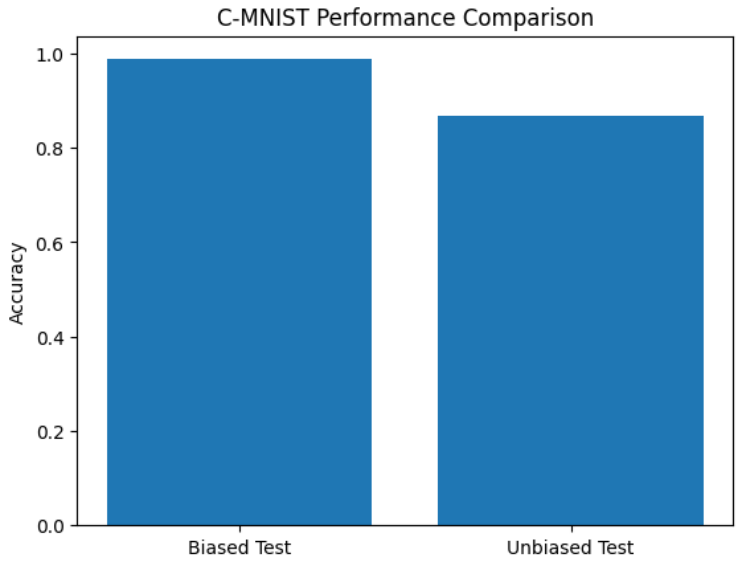

Figure 6: Biased vs Unbiased Accuracy

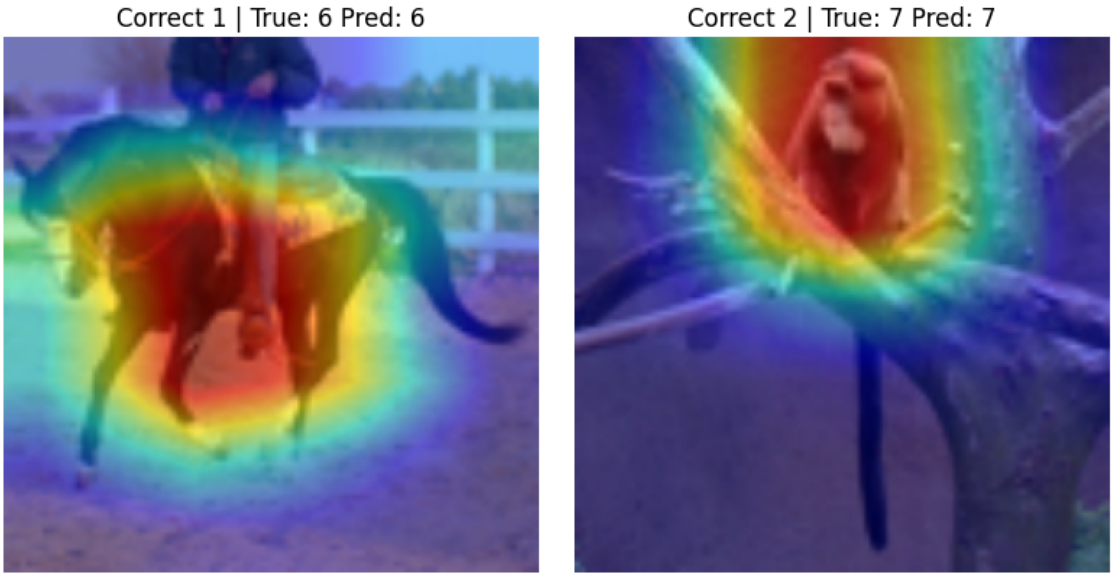

Figure 7: GradCAM - Correct Predictions

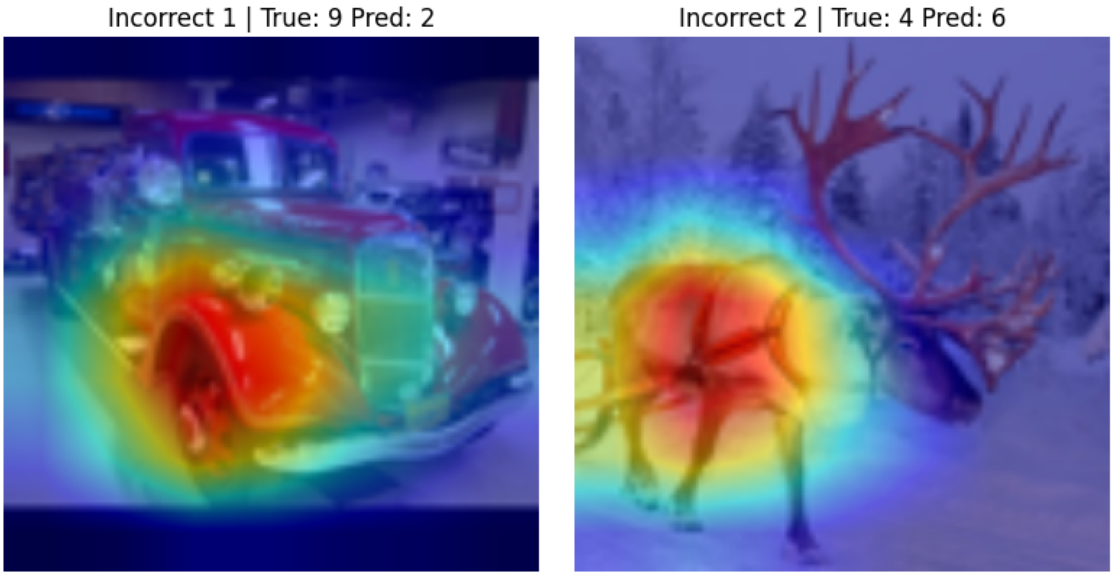

Figure 8: GradCAM - Incorrect Predictions